

LAB REPORT 02 - ADVERSARIAL ATTACKS

In this lab, we explored the topic of adversarial attacks, which means those kinds of perturbations applied to images such that the human eye is usually unable to see any difference, while the machine produces a misclassification error. The perturbation is crafted deliberately and mathematically, not random noise.

It is a very interesting field of study, and also a relevant one today, particularly for privacy concerns and protection from mass surveillance. For instance, I found myself getting interested in the work of researchers from the University of Chicago, who produced a tool called Fawkes, which adds imperceptible perturbations to personal photos before uploading them online, with the goal of making facial recognition systems fail to identify the person. A similar tool called Glaze applies the same principle to protect artists' styles from being learned by AI models.

However, we cannot be fully certain that these tools work in all real-world scenarios. This is because the protection is validated against known, public models, while the actual models deployed by surveillance companies or scrapers are private and constantly being retrained. Moreover, one established way of making a model more robust is to incorporate successful adversarial examples directly into the training data, so the model learns to resist them. This creates what is effectively an arms race: protection tools improve, models adapt, and the cycle continues.

This connects to a fundamental distinction in adversarial attacks. What we did in this lab is a white-box attack, meaning we had full access to the model's architecture and weights, which allowed us to compute the exact gradient of the loss with respect to the input pixels. That is precisely what makes FGSM possible in its pure form. Real-world scenarios like Fawkes are instead closer to a black-box setting: the attacker knows nothing about the target model's internals and can only observe its outputs. The reason researchers believe protection tools can still work in this setting is transferability: adversarial perturbations crafted against one model sometimes fool a different model too, particularly when the two share similar architecture or training data. It is not guaranteed, but it is frequent enough to be exploitable. This is the somewhat fragile assumption that tools like Fawkes are ultimately built on.

Coming back to this specific notebook, the attack we implemented is called FGSM (Fast Gradient Sign Method), introduced by Goodfellow et al. in 2014. The core idea is elegant: rather than changing pixels randomly, you ask the model itself to tell you which direction to push each pixel to maximise its own error. Concretely, you compute the gradient of the loss with respect to the input image, take only its sign (either +1 or -1 per pixel), and add a small step of size epsilon (ϵ) in that direction.

Something interesting about the story behind the paper is that Goodfellow reportedly wrote the code in a single night, and the idea came out of a casual discussion. At the time, a widespread belief was that adversarial attacks were possible because neural networks are highly nonlinear, and therefore somehow fragile in complex, hard-to-predict ways. The paper, "Explaining and Harnessing Adversarial Examples" (Goodfellow, Shlens and Szegedy, Google Inc., 2014), was written precisely to challenge that view. Goodfellow argued instead that the vulnerability comes from the opposite reason: the very linearity of the model, taken

into an extremely high-dimensional space. In high dimensions, even a tiny push in a consistent direction across thousands of pixels accumulates into a large effect on the output, enough to flip the classification entirely. It is a counterintuitive but compelling argument, and it reframes adversarial attacks not as a bug of complex models, but as an almost inevitable consequence of how linear operations scale.

The model we attacked was a LeNet-style CNN pretrained on MNIST, loaded from a local checkpoint. Before applying any attack, we ran a baseline accuracy check to verify it was working correctly. MNIST is the same dataset we used in the deep learning course, made of 10 handwritten digits, each with natural variation in how different people write them.

Given how FGSM works, it follows logically that the higher the epsilon, the larger the perturbation applied to each pixel, and therefore the more the adversarial image diverges from the original. The interesting challenge, however, is finding a value of epsilon that is large enough to fool the model but small enough that a human looking at the image would notice nothing unusual. This is the core tension of adversarial attacks.

A line of code I found particularly interesting is `images.requires_grad = True`. Normally in a neural network, we compute gradients with respect to the weights, that is what training is. Here we do something unusual: we compute the gradient with respect to the input pixels themselves, because what we want to modify is not the model but the image.

Playing with the interactive explorer, I found several interesting cases. On sample 2325, a successful adversarial attack occurred at epsilon = 0.3, where the model predicted the digit 2 instead of the true label 7, with a confidence of 26.1%. This confusion is not entirely surprising: a heavily perturbed 7 can start to resemble a 2 structurally, particularly when the horizontal stroke gets distorted. It is also worth noting that the model's confidence in its wrong answer was quite low, which suggests it was not strongly "convinced" — the attack pushed it over the decision boundary, but only just.

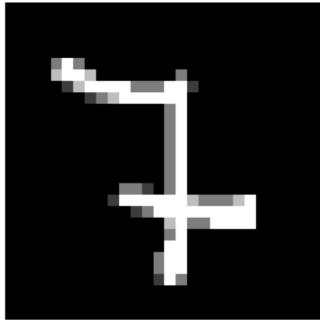
At epsilon = 0.3 the perturbation also starts to become visible to the human eye, as we can see in the image above: the adversarial version of the 7 looks noisy and distorted in a way the original does not. At lower epsilon values, around 0.05 to 0.1, the two images are essentially indistinguishable to me visually, which is precisely the more interesting and concerning regime for real-world applications.

digit 7 → exchanged by 2

Interactive Adversarial Examples Explorer

Sample Index: 2325
 Epsilon (ϵ): 0.300

Original Image
 True Label: 7 | Predicted: 7 (confidence: 100.0%)



Adversarial Image ($\epsilon=0.300$) / ATTACK SUCCEED
 Predicted: 2 (confidence: 26.1%)



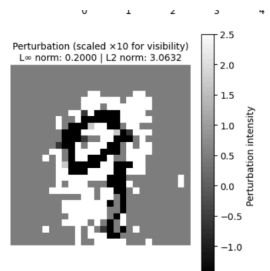
More interesting is the case of digit 9, sample 3286, which was misclassified as a 4 at a lower epsilon of 0.2, with a confidence of 56.6%. This is notable for two reasons. First, the attack succeeded at a smaller perturbation budget than the previous example, suggesting this particular 9 sits closer to the model's decision boundary. Second, the confusion between 9 and 4 is not arbitrary: both digits share a closed upper region and a descending vertical stroke, so there is a geometric logic to the misclassification. Looking at the adversarial image, it still reads unmistakably as a 9 to the human eye, which illustrates the core property of FGSM: the perturbation is calibrated to cross the model's decision boundary while remaining below the threshold of human perception. The perturbation map is also revealing here — the highest intensity pixels cluster around the loop and stroke of the 9, which are precisely the features that make it recognisable. FGSM is not adding noise randomly; it is targeting the regions the model itself finds most informative

Sample Index: 3286
 Epsilon (ϵ): 0.300

Original Image
 True Label: 9 | Predicted: 9 (confidence: 100.0%)



Adversarial Image ($\epsilon=0.200$) / ATTACK SUCCEEDED
 Predicted: 4 (confidence: 56.6%)



To better illustrate how robustness varies across digits, I tried to look for the most robust and the most fragile examples I could find. The digit 2 proved to be the most robust: even at epsilon 0.3, the maximum perturbation budget used across all experiments, the attack failed entirely, and the model continued to predict 2 with 37.1% confidence. At this epsilon level the adversarial image is visibly destroyed to the human eye, heavily noisy and barely recognisable as anything, yet the model did not flip its prediction. The L₂ norm of 5.2978 is the highest recorded across all examples, meaning FGSM applied the largest total perturbation energy of any case tested and still could not cross the decision boundary. This likely reflects the geometric distinctiveness of the 2: its curved top, diagonal stroke and flat

base make it sufficiently different from every other digit that even aggressive perturbation cannot push it convincingly toward another class.

The digit 1 tells the opposite story entirely. Sample 2997 was misclassified as a 4 at epsilon 0.19, the lowest epsilon at which I found a successful attack across all experiments, with a confidence of just 18.1%. The probability distribution after the attack is almost flat, with probability spread thinly across many classes, which suggests the model is not so much convinced it is a 4 as it is simply lost. This makes intuitive sense: a 1 is geometrically the simplest digit in MNIST, just a near-vertical stroke with very few active pixels. FGSM has little surface area to work with, but precisely because the digit is so minimal, even a small perturbation can make it ambiguous. A slightly altered vertical stroke could plausibly resemble a 1, a 4, a 7, or even an i, and the model has very little additional structure to anchor its prediction to.

Together these two cases illustrate that robustness under FGSM is not uniform and reflects two distinct factors: how geometrically distinctive the digit is, which determines how far it sits from the boundaries of other classes, and how much visual information the digit contains, which determines how much the model has to hold onto when perturbation is applied.

